Cyborg Development Workflow

Task-by-Task Implementation & Architecture Mapping • V1.0

This document outlines the sequential, eight-task workflow required to construct the Cyborg Supermarket ecosystem. Each task is isolated to a single development stage, ensuring that infrastructure, operations, and AI features are built systematically without architectural overlap.

1. Foundation Setup & Config → **2. Data Infra** Core APIs → **3. Store Ops** POS Client → **4. Customer** Mobile App

STAGE: FOUNDATION

Task 1: Environment Setup & Architecture Initialization

Action: Initialize Git repositories and define the Clean Architecture folder structures. Configure environment variables and prepare the cloud hosting infrastructure.

System Link: Scaffolds the **API Gateway** and establishes connections to the **Data Layer** (PostgreSQL/Redis).

```
Backend: src/config/ (Envs), src/main.ts (Entry)
Mobile: lib/app/ (Themes), lib/core/network/ (Dio client)
```

STAGE: DATA INFRASTRUCTURE

Task 2: Database Modeling & Core API

Action: Construct the relational database schema (Users, Products, Inventory, Orders). Develop the foundational REST/gRPC backend routes for CRUD operations and JWT authentication.

System Link: Builds out the **Customer & Auth Service**, **Inventory Service**, and **Orders Service**.

```
Backend: src/database/ (Migrations, ORM Models)
Backend: src/modules/auth/, src/modules/inventory/
```

5. Intelligence
AI Integrations
→
6. Unification
Cloud-Local Sync
→
7. QA
Testing
→
8. Launch
Deployment

STAGE: STORE OPERATIONS

Task 3: Desktop POS & Cashier System

Action: Develop the physical store checkout interface for Windows. Implement barcode scanning hardware hooks, local cart management, and shift tracking mechanisms to ensure offline resilience.

System Link: Focuses purely on the **Windows Desktop POS** client and its dedicated **Local SQLite Database** for zero-downtime operation.

```
POS: Local Views (Scanning, Checkout, Shift Summary)
Backend: src/modules/orders/ (Update to accept POS sync)
```

STAGE: CUSTOMER EXPERIENCE

Task 4: Flutter Mobile App Interface

Action: Build the cross-platform frontend using Flutter. Implement the digital product catalog, customer registration, shopping cart UI, and the shared "Family Cart" budgeting interface.

System Link: Connects the **Flutter Mobile App** presentation layer directly to the API Gateway to consume standard inventory and order endpoints.

```
Mobile: lib/features/product_catalog/ (UI & BLoC)
Mobile: lib/features/family_cart/ (Budgeting logic)
```

STAGE: INTELLIGENCE INTEGRATION

Task 5: AI Engine & Smart Features

Action: Embed the platform's unique selling propositions. Configure the Egyptian Arabic NLP for voice orders, program the Smart Pantry tracking logic, and implement vector embeddings for dynamic product substitutions.

System Link: Activates the **AI & Voice Engine** backend service and connects the mobile app's microphone state to the processing pipeline.

```
Backend: src/modules/ai-engine/ (NLP parsing logic)
Mobile: lib/features/ai_assistant/ (Voice recording UI)
```

STAGE: SYSTEM UNIFICATION

Task 6: Cloud-Local Synchronization

Action: Bridge the physical POS systems with the cloud backend. Implement a bidirectional delta-sync engine so mobile orders instantly reflect as reserved inventory on the cashier's screen, and physical sales deduct online stock.

System Link: Deploys the **Event Bus (Redis)** and establishes WebSocket/gRPC pipelines between the Gateway and POS clients.

```
Backend: src/modules/inventory/ (WebSocket Gateways)
POS: Background Sync Engine (SQLite to Cloud DB delta)
```



STAGE: QUALITY ASSURANCE

Task 7: QA Testing & Security Audit

Action: Conduct rigorous stress testing. Simulate concurrent orders to verify Optimistic Locking on inventory. Audit RBAC controls, API rate limits, and ensure POS offline queueing behaves correctly during simulated outages.

System Link: Validates the **API Gateway** security, database locking mechanisms, and frontend error handling.

```
Backend: src/modules/*/tests/ (Unit & e2e testing)
Mobile: test/features/ (Widget & Integration tests)
```

STAGE: LAUNCH

Task 8: Deployment & Live Production

Action: Finalize infrastructure provisioning. Deploy the containerized backend to production clusters. Publish the mobile applications to app stores and install the compiled POS binary on store hardware.

System Link: Transitions all architectural components from development environments to active live domains.

```
Root: docker-compose.yml, Dockerfiles
Root: .github/workflows/ (CI/CD Pipelines)
```

App Store Release

Submit iOS build to TestFlight & Android to Play Console for review.

POS Hardware Prep

Flash Windows machines, pair Bluetooth scanners, and configure thermal printers.

Database Seeding

Run production migrations and seed the initial supermarket inventory catalog.

Monitoring Setup

Activate logging dashboards to track API health and AI inference latency.